ГУАП

КАФЕДРА № [ВПИСАТЬ КАФЕДРУ]

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

_____	_____	_____
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ № 2
ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ ПРОВЕРОК ПРОГРАММНОГО ПРОДУКТА

по МДК.03.02 «Обеспечение качества функционирования компьютерных систем»

РАБОТУ ВЫПОЛНИЛ

Студент гр. №
[ВПИСАТЬ ГРУППУ] _____ _____ _____

	подпись, дата	Д. Карачебан

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3

1 Сведения о работе и проекте.....4

2 Постановка задачи.....5

3 Выполнение работы.....7

3.1 Проектирование проверок.....7

3.2 Данные и автоматизация.....9

3.3 Риски и приоритеты.....9

4 Проверка результатов.....13

4.1 Фактические прогоны.....13

4.2 Покрытие и принятые решения.....16

4.3 Семь компилируемых дефектов и восстановление.....17

5 Индивидуальный вклад и взаимодействие.....24

ЗАКЛЮЧЕНИЕ.....25

Приложение Б — модульное подтверждение.....26

СПИСОК ИСПОЛЬЗОВАННЫХ МАТЕРИАЛОВ.....27

ВВЕДЕНИЕ

Цель работы — разработать и практически проверить систему контроля качества собственного модуля IAM в проекте HitPipe. Объект проверки — чистое правило пригодности прикладной сессии и прикладной сценарий, использующий внешние зависимости через порты.

Задачи: вывести ожидаемые результаты из требований, выбрать техники проектирования тестов, подготовить воспроизводимые данные, выполнить локальные и CI-прогоны, проанализировать покрытие и проверить чувствительность набора внесением отдельных компилируемых дефектов.

1 СВЕДЕНИЯ О РАБОТЕ И ПРОЕКТЕ

Поле	**Значение**
Автор	Дмитрий Карачебан
Проект	HitPipe — сопровождение бизнес-операций между цифровыми сервисами
Индивидуальный участок	IAM: проверка прикладной сессии; каталог services/iam
Роль в команде	Руководитель проекта; аутентификация, авторизация и профиль
Рецензент	Тимофей Патрушев
Период работы	18–19 октября 2026 года
Репозиторий	hitpipe/hitpipe
Интегрированная версия	main, commit 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00; PR №12–19 объединены 19.10.2026

2 ПОСТАНОВКА ЗАДАЧИ

Проверяются SessionPolicy.Evaluate и AuthenticateSession.Service.Execute. Потребитель использует SessionLookup и Clock; для воспроизведения предусмотрены файловая реализация и фиксированная подмена. Источники ожидаемых значений — HP-BP-001/T/002 –003, T/013, HP-BP-007/T/005 и уточнение R2, записанное до реализации. Ожидания не вычисляются повторным вызовом проверяемой функции.

Основные правила: неизвестная, отозванная, истёкшая или некорректная сессия не признаётся действующей; равенство срока означает истечение; абсолютная продолжительность не превышает 24 часов; недоверенное время не позволяет получить Valid; Lookup выполняется до Clock; ошибка и отмена не превращаются в запасной успех. Диагностика не раскрывает входные данные и секреты.

Требование или риск	**DK-FR-01/02**
Проверяемое свойство	Нормальное решение, отзыв, неизвестность, обе границы срока и приоритет причин
Уровень	Предметные табличные тесты

Требование или риск	**DK-FR-03**
Проверяемое свойство	Подмена, ошибки, отмена, отсутствие зависимостей и порядок чтения времени
Уровень	Прикладные контрактные тесты

Требование или риск	**DK-FR-04, DK-NFR-02**
Проверяемое свойство	Строгая конфигурация, безопасные коды, версия и отсутствие утечек

Требование или риск	**DK-FR-04, DK-NFR-02**
Уровень	CLI-тесты и запуск бинарника

Требование или риск	**DK-NFR-01**
Проверяемое свойство	Детерминизм, неизменность входа и отсутствие ввода-вывода в domain
Уровень	Тесты инвариантов и анализ импортов

Требование или риск	**Риск нечувствительных проверок**
Проверяемое свойство	Изменённое предметное правило вызывает отказ тестов
Уровень	Мутационные опыты и отрицательный автоматический контроль

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Проектирование проверок

Применены три основные техники: классы эквивалентности, граничные значения и таблица решений. Дополнительно используются подмена зависимостей, проверка инвариантов и направленный нечёткий поиск входов. Это позволяет проверять не только строки исходников, но и конкретные обязательства контракта.

Класс проверки	**Классы эквивалентности**
Вход и независимое ожидание	Известная неотозванная сессия с корректными датами и доверенным временем до сроков → session_valid; unknown и revoked → Denied
Реальные тесты	TestEvaluateValid, TestEvaluateUnknown, TestEvaluateRevoked

Класс проверки	**Граничные значения**
Вход и независимое ожидание	Для idle и absolute: за 1 нс до срока допустимо, ровно на сроке и через 1 нс — истекло; 24 часа и превышение на 1 нс; нулевые и непредставимые даты
Реальные тесты	TestEvaluateIdleBoundary, TestEvaluateAbsoluteBoundary, TestEvaluateInvalidChronology, TestEvaluateClockAndFutureCreation

Класс проверки	**Таблица решений**
Вход и независимое ожидание	Несколько одновременно истинных условий: clock → unknown → revoked → malformed → future → absolute → idle → valid
Реальные тесты	TestDecisionTablePrecedence

Класс проверки	**Подмена за одним портом**
Вход и независимое ожидание	Один Execute и File/Fixed: normal, error, idle_equal, unknown, revoked; ожидаемые значения из контракта
Реальные тесты	TestSameConsumerContractWithBothAdapters

Класс проверки	**Ошибки и свежесть**
Вход и независимое ожидание	Ошибка Lookup → недоступность без обращения к Clock; Clock читается после Lookup; отмена до или во время порта → недоступность
Реальные тесты	TestPortErrorsDoNotAllowOrLeak, TestClockReadAfterLookupAndCancellation

Класс проверки	**Недоверенный JSON**
Вход и независимое ожидание	Дубли, лишний или неверно написанный ключ, null, неверный тип/версия, лишний хвост, глубина, размер и неверная дата → ошибка конфигурации без утечки
Реальные тесты	TestDiagnosticMalformedAndCanary, TestAdapterFailureAndBoundaryInputs

Класс проверки	**Инварианты**
Вход и независимое ожидание	Valid требует известности, отсутствия отзыва, доверенного времени и корректных сроков; одинаковый вход даёт одинаковый результат
Реальные тесты	FuzzValidityInvariants, TestEvaluateDeterministicImmutable

Краткий контрольный список: штатный исход; каждый запрещающий класс; две стороны и точное равенство срока; конфликт причин; обе реализации зависимости; ошибка и отмена; неизменность данных; безопасный

вывод; восстановление после негативного опыта. Конкурентные тесты и race относятся к реально исполненным сценариям и не доказывают безопасность любых будущих адаптеров.

3.2 Данные и автоматизация

Используются синтетические даты и логические признаки. Входные файлы включены в репозиторий; сброс состояния не требует доступа к внешним системам. Для воспроизведения нужны Go 1.27.2, Python 3.10 или новее и C-компилятор для детектора гонок.

Команды выполняются из корня репозитория:

```
python3 tools/pr12/workspace.py validate
PR12_RACE=1 python3 tools/pr12/workspace.py check iam --require
python3 tools/pr12/workspace.py smoke iam --require
go -C services/iam test -coverpkg=./... -coverprofile=iam-all.cover ./...
```

Обёртка требует непустой набор именованных тестов и фактические успешные события; пропуск и отсутствие реализации не считаются успешной проверкой. Workflow .github/workflows/pr12-dk.yml запускает анализ, тесты, race, dev/release, покрытие и семь мутаций, затем сохраняет артефакты. Профиль -coverpkg=./... учитывает вызовы адаптеров из тестов CLI.

Скрытые дефекты преподавателя не выдавались; в число выполненных опытов включены только собственные входы и сохранённые мутационные опыты.

3.3 Риски и приоритеты

Использована качественная матрица методических указаний. При низкой вероятности малый ущерб даёт низкий риск, значимый — средний, тяжёлый — высокий. При средней или высокой вероятности малый ущерб даёт средний, значимый и тяжёлый — высокий. Это экспертная оценка причин и последствий, а не статистика реальных инцидентов.

Риск	**DK-RISK-01, безопасность**
Событие, причина и последствие	Unknown/revoked/expired получает Valid из-за ошибки границы или допуска по умолчанию; возможно неправомерное использование сессии.
Оценка и основание	Вероятность средняя: несколько ветвей и равенство срока. Ущерб тяжёлый; приоритет высокий.
Проверка, частота и остаток	Каждый PR: boundary, unknown/revoked и M01–04. Разрешение бизнес-действия проверяется отдельно.

Риск	**DK-RISK-02, безопасность**
Событие, причина и последствие	Неоднозначный JSON принят или раскрыт в ошибке; причина — дубли, регистр ключа или стандартный текст парсера.
Оценка и основание	Вероятность средняя: естественный дефект регистра действительно найден. Ущерб тяжёлый; приоритет высокий.
Проверка, частота и остаток	Каждый PR: строгий разбор, canary, размер/глубина/ошибочные типы. Защита настоящего транспорта проверяется при его подключении.

Риск	**DK-RISK-03**
Событие, причина и последствие	После задержки используется прежнее время или сбой маскируется успехом; возможно действие после истечения срока.
Оценка и основание	Вероятность средняя: порядок обращений и запасные значения легко нарушить. Ущерб тяжёлый; приоритет высокий.

Риск	**DK-RISK-03**
Проверка, частота и остаток	Каждый PR: Lookup→Clock, задержка, ошибки/отмена и M06. Для реального источника нужны отдельные проверки свежести и отказов.

Риск	**DK-RISK-04**
Событие, причина и последствие	time.Now принят за доверенное наблюдение без проверки исправности часов; неверная пригодность сессии.
Оценка и основание	Вероятность средняя: системные часы доступны без подтверждения. Ущерб тяжёлый; приоритет высокий.
Проверка, частота и остаток	Каждый PR: nil/false HealthProbe и отказ CLI system. Реальная проверка часов и скачков времени относится к адаптеру эксплуатации.

Риск	**DK-RISK-05**
Событие, причина и последствие	Бинарник не соответствует исходникам либо зависит от скрытого окружения; воспроизведение срывается.
Оценка и основание	Вероятность средняя: несколько веток и сред. Ущерб значимый; приоритет высокий.
Проверка, частота и остаток	Каждый PR: source metadata, dev/release и запуск копии вне исходников. Целевые платформы дополнительно проверяются перед поставкой.

Риск	**DK-RISK-06**
Событие, причина и последствие	Зелёные тесты не проверяют предметное правило из-за слабого утверждения или пропуска.

Риск	**DK-RISK-06**
Оценка и основание	Вероятность средняя: покрытие строк не означает полноту поведения. Ущерб значимый; приоритет высокий.
Проверка, частота и остаток	Каждый PR: непустые именованные тесты и мутации; пропуск дефекта приводит к новому тесту и повтору. Ожидания сопоставляются с текстом контракта.

Риск	**DK-RISK-07**
Событие, причина и последствие	Изменяемый адаптер разделяет данные между вызовами, вызывая гонку и нестабильное решение.
Оценка и основание	Вероятность низкая: текущие входы — значения, потребитель не хранит состояние. Ущерб значимый; приоритет средний.
Проверка, частота и остаток	Race, проверки владения и параллельных чтений. Настоящее общее хранилище потребует нагрузочных и конкурентных интеграционных тестов.

Риск	**DK-RISK-08**
Событие, причина и последствие	Подпись или изображение относится к иной версии, таблица теряет смысл при переносе.
Оценка и основание	Вероятность средняя: четыре отчёта и множество иллюстраций. Ущерб небольшой; приоритет средний.
Проверка, частота и остаток	При изменении отчётов: проверка SHA, подписей, границ и реальный PDF-экспорт; изображения сверяются с источниками.

4 ПРОВЕРКА РЕЗУЛЬТАТОВ

4.1 Фактические прогоны

Запуск	**Финальная локальная проверка C3**
Версия и условия	5b14ab92d1ca2a68f46fd3bda783b188549fbf5a; чистое дерево; Go 1.27.2; GOMAXPROCS=2, GOFLAGS=-p=1
Результат	PASS, exit 0: vet, именованные тесты и race; пять пакетов.

Запуск	**Межпакетное покрытие**
Версия и условия	Та же версия, -coverpkg=./...
Результат	PASS: 94,5% операторов во всём модуле.

Запуск	**Запуск бинарников**
Версия и условия	Та же версия; dev/release, копия вне исходников
Результат	PASS: штатный вход, ошибочный JSON и неверный формат.

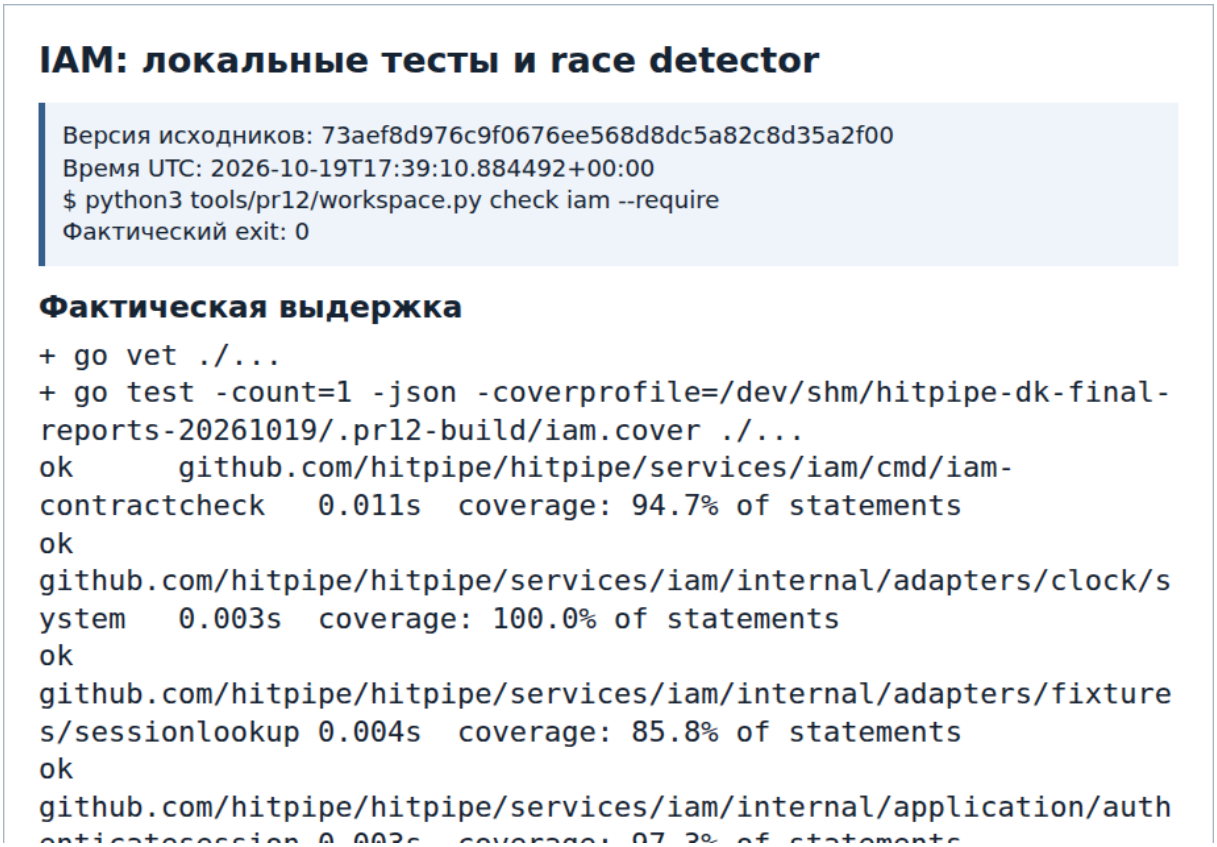
Запуск	**Fuzz-проверка**
Версия и условия	61a54285e979fd373eb101116ecb6fa1531913f5; та же предметная логика
Результат	PASS: 215 557 выполнений за 3 секунды, два рабочих процесса. Поиск не является исчерпывающим.

Запуск	**GitHub CI проверенной версии**
Версия и условия	2cd7c032e8ede98c75181599b440e97657bff197; исходники и тесты совпадают с локальной версией

Запуск	**GitHub CI проверенной версии**
Результат	SUCCESS, run 39177171258: race, запуск, покрытие, все семь мутаций и восстановление.

Запуск	**Повтор на интегрированном main**
Версия и условия	73aef8d976c9f0676ee568d8dc5a82c8d35a2f00, 19.10.2026
Результат	PASS: локальные проверки и диагностические запуски; четыре обязательные проверки main — SUCCESS.

Источники и тесты IAM не изменились между сохранённой C3-версией и интегрированным main. Это проверено сравнением поддерева services/iam. Исторические измерения сохраняют собственные версии; они не переименованы в новые измерения.



IAM: локальные тесты и race detector

Версия исходников: 73aef8d976c9f0676ee568d8dc5a82c8d35a2f00
Время UTC: 2026-10-19T17:39:10.884492+00:00
$ python3 tools/pr12/workspace.py check iam --require
Фактический exit: 0

Фактическая выдержка

```
+ go vet ./...
+ go test -count=1 -json -coverprofile=/dev/shm/hitpipe-dk-final-
reports-20261019/.pr12-build/iam.cover ./...
ok      github.com/hitpipe/hitpipe/services/iam/cmd/iam-
contractcheck   0.011s  coverage: 94.7% of statements
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/clock/s
ystem   0.003s  coverage: 100.0% of statements
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/fixture
s/sessionlookup 0.004s  coverage: 85.8% of statements
ok
github.com/hitpipe/hitpipe/services/iam/internal/application/auth
```

Рисунок 1 - Локальный прогон собственных IAM-тестов (часть 1 из 2)

```
enticatesession 0.003s  coverage: 97.3% of statements
ok
github.com/hitpipe/hitpipe/services/iam/internal/domain/sessionpo
licy    0.004s  coverage: 100.0% of statements
+ go test -race -count=1 ./...
ok      github.com/hitpipe/hitpipe/services/iam/cmd/iam-
contractcheck   1.027s
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/clock/s
ystem   1.012s
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/fixture
s/sessionlookup 1.018s
ok
github.com/hitpipe/hitpipe/services/iam/internal/application/auth
enticatesession 1.012s
ok
github.com/hitpipe/hitpipe/services/iam/internal/domain/sessionpo
licy    1.014s
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете.

Рисунок 1 - Локальный прогон собственных IAM-тестов (часть 2 из 2)

GitHub Actions: проверки IAM и семь мутаций

> Версия исходников: b1c3ae8ac10a885c6f3cc417d1f52af850580516

Оригинальный журнал job 118166291075

```
2026-10-19T16:19:11.9040998Z + go test -race -count=1 ./...
2026-10-19T16:19:38.9149081Z M01 KILLED
2026-10-19T16:19:39.9145458Z M02 KILLED
2026-10-19T16:19:40.9249772Z M03 KILLED
2026-10-19T16:19:41.9419883Z M04 KILLED
2026-10-19T16:19:42.9615915Z M05 KILLED
2026-10-19T16:19:43.8782006Z M06 KILLED
2026-10-19T16:19:44.9800410Z M07 KILLED
2026-10-19T16:19:45.4935607Z All selected compiling behavioral
mutants killed; restored final gate PASS.
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете. Это журнал GitHub Actions от 16:19 UTC, а не новый локальный запуск. Job: github.com/hitpipe/hitpipe/actions/runs/39284442119/job/118166291075.

Рисунок 2 - Тесты IAM и мутационные проверки в CI

4.2 Покрытие и принятые решения

Межпакетный профиль iam-all.cover показывает 94,5% покрытия операторов модуля. Evaluate, Decision.IsValid и Service.Execute имеют 100% покрытия операторов в измеренном наборе. Это не означает покрытия всех ветвей, требований или распределённых условий эксплуатации.

Ограничение покрытия	**Значимость и решение**
main вызывает os.Exit	Вызов не исполняется внутри процесса unit-теста. Настоящий вход и коды завершения проверены отдельным запуском бинарника.
Защитные ошибки точки сборки	Некоторые ветви относятся к уже сконструированным зависимостям. Их назначение сохранено; проверки отсутствующих и типизированных nil-зависимостей добавлены отдельно.
Редкие варианты JSON и временных значений	Добавлены негативные, контрольные на утечки и регрессионные примеры. Полный перебор грамматики и всех отказов ОС в объём набора не входит.
Ошибки чтения и записи вывода	Существенные ветви дополнены регрессионными тестами; фактические результаты записаны в test-results.md.
Реальные часы, SQL, OIDC и отзыв в распределённой системе	Отсутствуют в проверяемом локальном модуле. Требуют интеграционных испытаний при добавлении соответствующих адаптеров; высокий процент покрытия не заменяет их.

Обычный профиль отдельного пакета не отражал вызовы адаптеров из CLI-тестов, поэтому добавлен межпакетный -coverpkg=./.... Решение

сохраняет видимость проверенной области и не суммирует проценты разных бинарников как единый показатель.

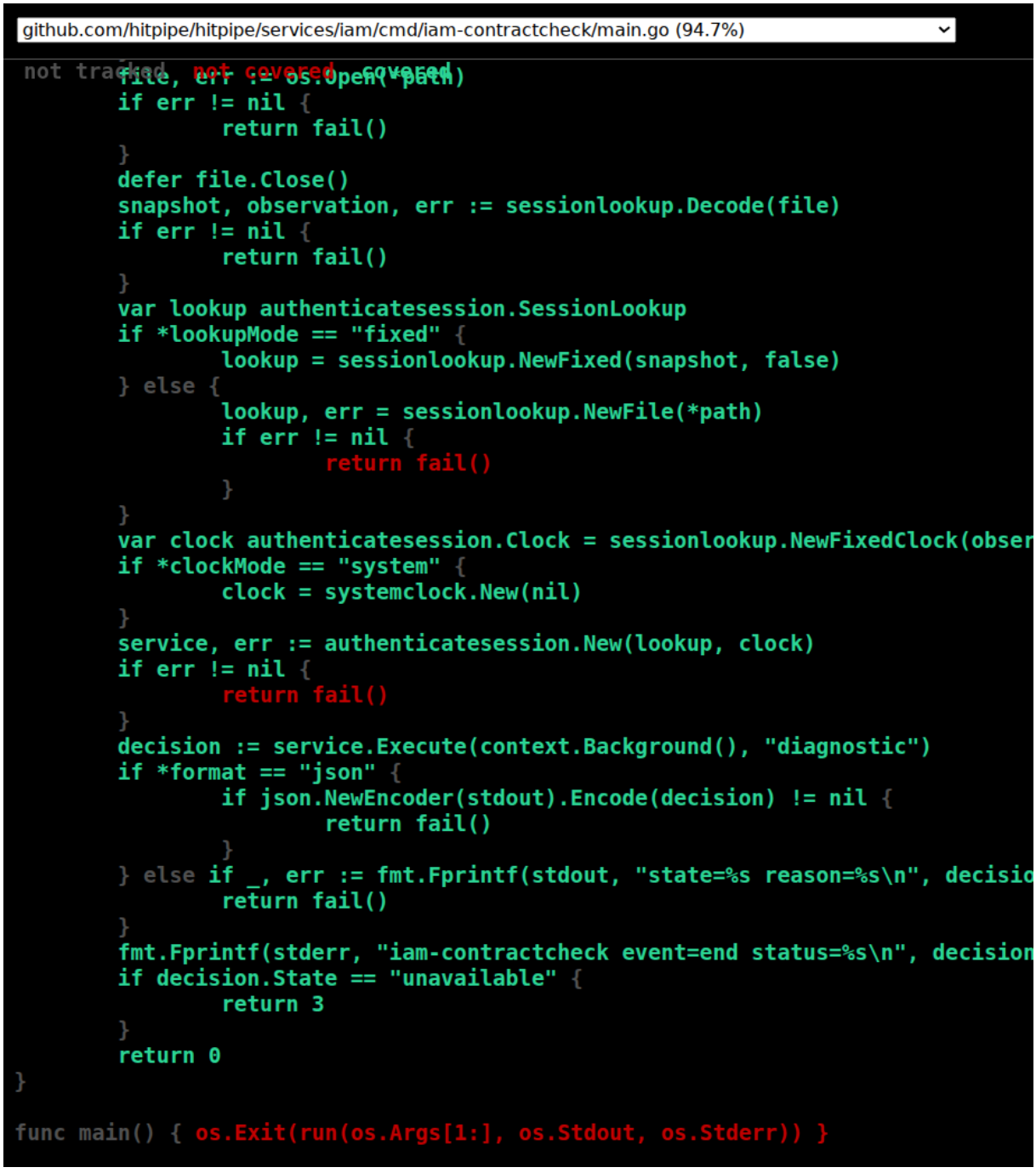


Рисунок 3 - Покрытие IAM и конкретный непроверенный участок

4.3 Семь компилируемых дефектов и восстановление

Каждая мутация вносилась отдельно во временном рабочем дереве. Последовательность опыта: исправный набор проходит → изменённый код компилируется → тот же набор запускается на мутанте → результат

анализируется → файл восстанавливается → исправный набор вновь проходит. Мутационные файлы не включены в рабочий код main.

Мутант и нарушенное правило	**M01: idle equality, > вместо >=**
Первый результат	Сборка 0; обнаружен TestEvaluateIdleBoundary/equal и другими тестами
Усиление и повтор	Повторный финальный CI также обнаружил дефект.

Мутант и нарушенное правило	**M02: absolute equality и приоритет**
Первый результат	Сборка 0; обнаружен TestEvaluateAbsoluteBoundary и таблицей решений
Усиление и повтор	Финальный CI обнаружил дефект.

Мутант и нарушенное правило	**M03: пропуск revoked**
Первый результат	Сборка 0; обнаружен TestEvaluateRevoked и прикладными/CLI-тестами
Усиление и повтор	Финальный CI обнаружил дефект.

Мутант и нарушенное правило	**M04: unknown получает допуск**
Первый результат	Сборка 0; обнаружен TestEvaluateUnknown и тестами подмены
Усиление и повтор	Финальный CI обнаружил дефект.

Мутант и нарушенное правило	**M05: удвоен предел срока жизни**
Первый результат	Сборка 0; обнаружен InvalidChronology/lifetime_above_cap
Усиление и повтор	Финальный CI обнаружил дефект.

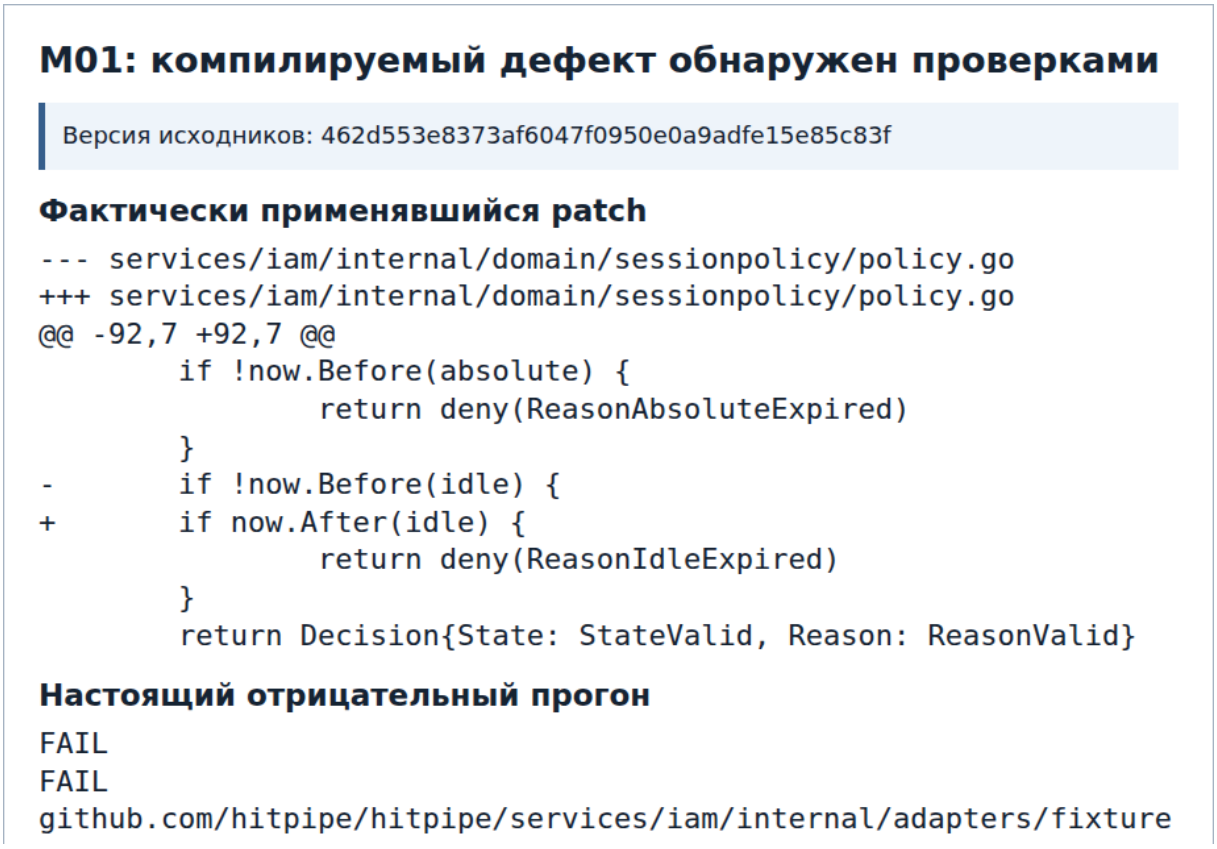
Мутант и нарушенное правило	**M06: ошибка Lookup превращается в допуск**
Первый результат	Сборка 0; обнаружен PortErrors/lookup и ошибочным сценарием подмены
Усиление и повтор	Финальный CI обнаружил дефект.

Мутант и нарушенное правило	**M07: IsValid проверяет только State**
Первый результат	Сборка 0; первый набор прошёл и пропустил ошибку
Усиление и повтор	Добавлен TestDecisionRejectsMixedStateAndReason. Повтор на 61a54285 завершился отказом теста; финальный CI обнаружил дефект.

Первый опыт выполнен на 462d553e8373af6047f0950e0a9adfe15e85c83f. Внесено семь дефектов: шесть обнаружены, один пропущен. После усиления обнаружены все семь. Некомпилируемых мутаций — ноль. Итоговые журналы подтверждают восстановленный исходный код и успешную финальную проверку.

M07 выявил содержательный пробел: Evaluate создаёт согласованные решения, однако вызывающий код способен вручную собрать противоречивую пару State/Reason. Новый тест проверяет, что IsValid не признаёт такую пару допустимой. Это усиление ожидания, а не проверка текста реализации.

Найден один естественный дефект диагностического декодера: ключ Version принимался вместо точного version. Регрессионный тест до исправления получил exit 0 вместо ожидаемого 2 и завершился отказом. После строгой проверки полей результат соответствует требованию. Дефект связан с HIT-19 и отличается от семи специально внесённых мутаций.



M01: компилируемый дефект обнаружен проверками

> Версия исходников: 462d553e8373af6047f0950e0a9adfe15e85c83f

Фактически применявшийся patch

```
--- services/iam/internal/domain/sessionpolicy/policy.go
+++ services/iam/internal/domain/sessionpolicy/policy.go
@@ -92,7 +92,7 @@
        if !now.Before(absolute) {
                return deny(ReasonAbsoluteExpired)
        }
-       if !now.Before(idle) {
+       if now.After(idle) {
                return deny(ReasonIdleExpired)
        }
        return Decision{State: StateValid, Reason: ReasonValid}
```

Настоящий отрицательный прогон

```
FAIL
FAIL
github.com/hitpipe/hitpipe/services/iam/internal/adapters/fixture
```

Рисунок 4 - Компилируемый дефект M01 обнаружен тестами (часть 1 из 2)

```
s/sessionlookup 0.004s
FAIL
FAIL
github.com/hitpipe/hitpipe/services/iam/internal/application/auth
enticatesession 0.007s
    policy_test.go:53: delta 0s: want idle_expired, got
{State:valid Reason:session_valid}
--- FAIL: TestEvaluateIdleBoundary/equal (0.00s)
FAIL
FAIL
github.com/hitpipe/hitpipe/services/iam/internal/domain/sessionpo
licy    0.003s
```

Восстановление исходного состояния

```
{
  "restored_worktree_clean": true,
  "final_clean_exit_code": 0
}
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете. Исторический эксперимент 18.10.2026. Дефект отсутствует в main; build/check/restore отражены в оригинальном protocol.json.

Рисунок 4 - Компилируемый дефект M01 обнаружен тестами (часть 2 из 2)

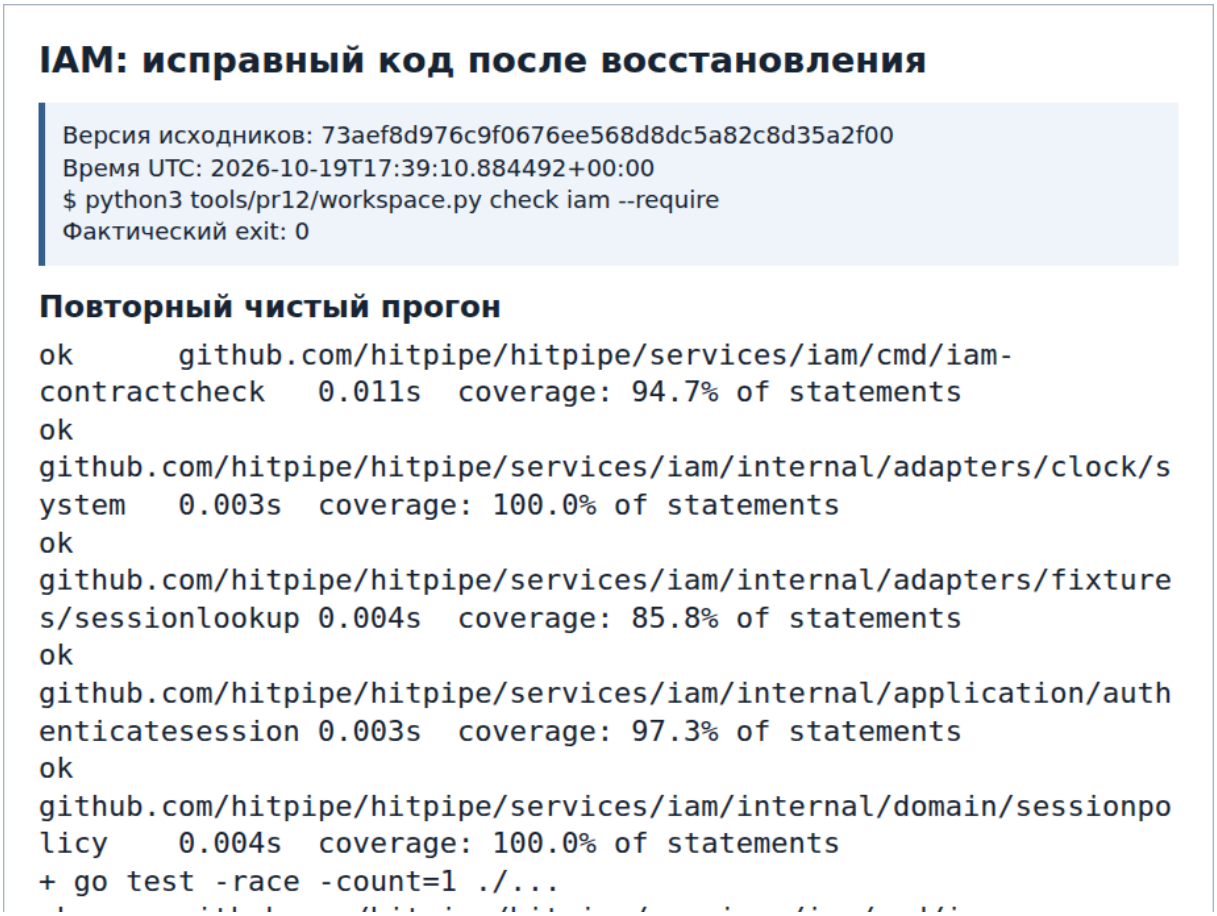


Рисунок 5 - Успешный прогон после восстановления (часть 1 из 2)

```
ok      github.com/hitpipe/hitpipe/services/iam/cmd/iam-
contractcheck   1.027s
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/clock/s
ystem   1.012s
ok
github.com/hitpipe/hitpipe/services/iam/internal/adapters/fixture
s/sessionlookup 1.018s
ok
github.com/hitpipe/hitpipe/services/iam/internal/application/auth
enticatesession 1.012s
ok
github.com/hitpipe/hitpipe/services/iam/internal/domain/sessionpo
licy    1.014s
```

Исходный протокол восстановления

```
{
  "restored_worktree_clean": true,
  "final_clean_exit_code": 0
}
```

Браузерный просмотр фактических файлов. Журнал показан выдержкой; полный оригинал и его SHA-256 сохранены в пакете. Новый прогон выполнен на интегрированном main; историческая запись восстановления указана отдельно.

Рисунок 5 - Успешный прогон после восстановления (часть 2 из 2)

5 ИНДИВИДУАЛЬНЫЙ ВКЛАД И ВЗАИМОДЕЙСТВИЕ

Индивидуальный участок Дмитрия Карачебана включает проектирование ожидаемых исходов IAM, предметные и прикладные тесты, строгие проверки парсера и вывода, анализ покрытия, изолированный запуск мутаций и реестр рисков. Общая обёртка проекта и закреплённые материалы используются как командное основание; тесты других сервисов не включаются в индивидуальные показатели IAM.

Рецензент участка — Тимофей Патрушев. Замечания к воспроизводимости команд и вёрстке таблиц устранены; код получил положительную оценку в пределах рассмотренного объёма. Результат объединён в основную ветку после обязательных проверок.

ЗАКЛЮЧЕНИЕ

Разработанный набор проверяет конкретные обязательства сессии, безопасное поведение зависимостей и диагностической команды. Применение нескольких техник обеспечило штатные, ошибочные и точные граничные случаи. Риски определили обязательные проверки при каждом изменении.

Покрытие 94,5% рассмотрено вместе с непроверенными свойствами. Мутационный опыт обнаружил слабое место первоначального набора; после нового регрессионного теста все семь компилируемых нарушений обнаруживаются. Восстановление исправного кода проверено. Результаты относятся к локальному модулю и задают основание для последующих интеграционных испытаний настоящего IAM.

ПРИЛОЖЕНИЕ Б — МОДУЛЬНОЕ ПОДТВЕРЖДЕНИЕ

Одно правило Evaluate используется одним Service.Execute. Потребитель владеет интерфейсами, а File/Fixed выбираются в точке сборки. Для обеих реализаций исполнены штатный, ошибочный и граничный случаи. Карта импортов не содержит циклов; владение значениями, свежесть времени и безопасные ошибки описаны в контракте и DK-ADR-001. Подробная модульная часть представлена в отдельном отчёте МДК01.04 ПР2.

СПИСОК ИСПОЛЬЗОВАННЫХ МАТЕРИАЛОВ

1. Методические указания к практическим работам №1 «Подготовка инструментального контура программного проекта» и №2 «Модульная архитектура» по МДК01.04. Актуальные редакции, предоставленные преподавателем.

2. Методические указания к практическим работам №1 «Требования и регламент» и №2 «Проектирование и реализация системы проверок программного продукта» по МДК03.02. Актуальные редакции, предоставленные преподавателем.

3. Проект HitPipe: описание MVP, регламент изменений, решение о технологическом стеке LAB-STACK-001; архитектурные материалы HP-BP-001, разделы T/002, T/003, T/013, и HP-BP-007, раздел T/005. Копии исходных документов и контрольные суммы — docs/labs/pr12/upstream и source-files.json.

4. Исходный код и документация IAM: module-contract.md, policy.go, service.go, адаптеры, диагностическая команда и тесты.

5. Материалы индивидуального участка Дмитрия Карачебана: требования, карта модулей, DK-ADR-001, проектирование тестов, результаты и реестр рисков.

6. Изменение требований, PR №13, реализация модуля, PR №14, итоги технического ревью, PR №19.

7. Успешный запуск CI для проверенной версии IAM. Журналы повторных запусков на интегрированной версии и исходные изображения включены в электронные материалы работы.